



VINUNIVERSITY

Lab 05: Stacks, Queues.

Week 04, Mar 25 2026

**College of Engineering and Computer Science,
Object-Oriented Programming & Data Structures Spring**

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your code and functions/modules should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables and functions should have meaningful names, and code should be organized into functions/methods where appropriate.
- Academic honesty is required in all work you submit to be graded. You should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions (modified to provide your student ID, your name, etc).
- You should **upload your .java file(s) to Codeforces before the end of the laboratory session** unless the instructor gave a specified deadline. **No Canvas or Codeforces submission** will get **0** point.
- Submit a separate .java file for each lab problem, with the exact filename specified in each lab question.
- Late submission of lab practice without an approved extension will incur the following penalties:
 1. No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
 2. The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
 3. The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.
- **Navigate to the Lab:** Open the specific link provided for this week's lab: <https://vinuni26oop.contest.codeforces.com/>.

Problem 1: Stack (Pre-Lab Assignment)

Note: Please complete this problem before arriving at the lab.

Write a Java program that converts a given positive decimal integer into its octal (base-8) equivalent using a stack data structure.

The program should repeatedly divide the number by 8, pushing the remainder onto the stack, and updating the number to the quotient. Once the number becomes 0, pop all elements from the stack to form the octal representation.

Input

A single positive integer N .

Output

A single string representing the octal equivalent of N .

Examples

Test Case 1:

Input:

33

Output:

41

Problem 2: Queue

Write a Java program that generates and prints the binary representation of all numbers from 1 to N using a queue data structure.

Instead of mathematically converting each number, use a Queue of Strings. Start by enqueueing "1". Then, repeatedly dequeue an element, print it, and enqueue two new elements by appending "0" and "1" to the dequeued string.

Input

A single integer N .

Output

The binary numbers from 1 to N , separated by a space.

Examples

Test Case 1:

Input:

5

Output:

1 10 11 100 101

Problem 3: Kth Largest Element using PriorityQueue

Write a Java program that finds the K -th largest element in a given sequence of numbers using a `PriorityQueue`.

You should use a Min-Heap (which is the default behavior of Java's `PriorityQueue`). Maintain the heap size at K . When inserting a new element, if the heap size exceeds K , remove the smallest element. The root of the heap will then represent the K -th largest element.

Input

The first line contains an integer K . The second line contains a sequence of space-separated integers.

Output

A single integer representing the K -th largest element in the sequence.

Examples

Test Case 1:

Input:

3

4 5 8 2 3 9 10 1

Output:

8

Problem 4: Implement a Queue using a Linked List

Write a Java program that implements a custom Queue using a Singly Linked List. You are **NOT** allowed to use Java's built-in `LinkedList` or `Queue` interfaces.

Create a custom `Node` class. Your queue must maintain `head` (front) and `tail` (rear) pointers to ensure `enqueue` and `dequeue` operations run in $O(1)$ time.

Input

An integer n , followed by n lines of operations: `enqueue x`, `dequeue`, `front`, or `empty`.

Output

The result of each `dequeue`, `front`, or `empty` operation. Output `null` if `dequeue`/`front` is called on an empty queue.

Examples

Test Case 1:

Input:

6

`enqueue 10`

`enqueue 20`

`front`

`dequeue`

`empty`

`dequeue`

Output:

10

10

false

20

Problem 5: Palindrome Linked List using a Stack

Check if a custom singly linked list is a palindrome. Traverse the linked list and push every node's value onto a standard Stack. Then, traverse the list from the head again, popping elements from the stack and comparing them to the current node's value.

Input

A single line of space-separated integers.

Output

YES if the linked list is a palindrome, and NO otherwise.

Examples

Test Case 1:

Input:

1 2 3 2 1

Output:

YES

Problem 6: Next Smaller Element (Optional / Advanced)

Find the next smaller element for each element in an array using a stack. The next smaller element of an element x is the first strictly smaller element to the right of x . If it does not exist, return -1 . Utilize a monotonic stack to achieve $O(n)$ time complexity.

Input

An integer n representing the size of the array, followed by the n elements separated by semicolons.

Output

An array of integers representing the next smaller elements, separated by semicolons.

Examples

Test Case 1:

Input:

5

4;8;5;2;25

Output:

2;5;2;-1;-1